

PRINTABLE MASTER ROADMAP

GenAI Mastery Canon

A clean printable guide for the complete GenAI roadmap: foundations, retrieval, evaluation, safety, agents, frameworks, capstones, cost, debugging, and hiring signal design.

22

modules across the full
canon

689h

estimated full mastery
path

449h

hiring-ready track with
capstone

10

mandatory build
checkpoints

Best path: start with the market-ready modules, build at every checkpoint, and package at least one serious capstone as hiring evidence.

Quick Navigation

Linked Documents

How To Use This Canon

What This Canon Covers

Execution Modes

Time Model

Mandatory Build Cadence

Phase Map

Recommended Market-Strong Stack Hidden Inside This Canon

Module 1: GenAI Landscape And Mental Models

- Topic 1.1: GenAI system taxonomy and vocabulary
- Topic 1.2: Anatomy of a GenAI application
- Topic 1.3: Failure modes and thinking patterns

Module 2: Transformer And Modern LLM Internals

- Topic 2.1: Text processing, tokens, and context
- Topic 2.2: Transformer mechanics
- Topic 2.3: From pretraining to instruction following

Module 3: Prompting And Structured Generation

- Topic 3.1: Prompt design patterns
- Topic 3.2: Structured output and schema-driven generation
- Topic 3.3: Prompt debugging and prompt systems

Module 4: Embeddings And Semantic Representations

- Topic 4.1: Embedding concepts and vector geometry
- Topic 4.2: Embedding model selection and evaluation
- Topic 4.3: Embedding pipelines and chunk representations

Module 5: Vector Search And Vector Datastores

- Topic 5.1: Similarity search fundamentals

- Topic 5.2: Vector database ecosystem
- Topic 5.3: Filtering, hybrid retrieval, and scale tradeoffs

Module 6: RAG Foundations

- Topic 6.1: Ingestion and preprocessing
- Topic 6.2: Retrieval pipeline basics
- Topic 6.3: Answer generation with citations

Module 7: Advanced Retrieval Engineering

- Topic 7.1: Chunking, metadata, and hierarchical retrieval
- Topic 7.2: Query transformation and reranking
- Topic 7.3: Advanced RAG patterns

Module 8: Evaluation, Observability, And Experimentation

- Topic 8.1: Retrieval and generation metrics
- Topic 8.2: Test sets, judges, and regression systems
- Topic 8.3: Tracing and production observability

Module 9: Safety, Guardrails, And Reliability

- Topic 9.1: Input and output safety
- Topic 9.2: Tool and retrieval security
- Topic 9.3: Reliability engineering for LLM apps

Module 10: Agent Fundamentals

- Topic 10.1: What agents are and are not
- Topic 10.2: Tool use, planning, and memory
- Topic 10.3: Agent architectures and failure handling

Module 11: LangChain Core

- Topic 11.1: LangChain core abstractions
- Topic 11.2: Retrieval, tools, and agents in LangChain
- Topic 11.3: Production use of LangChain

Module 12: LangGraph Mastery

- Topic 12.1: Graph mental models and state design
- Topic 12.2: Durable execution, persistence, and interrupts

Topic 12.3: Production graph patterns

Module 13: Model Context Protocol (MCP)

Topic 13.1: MCP protocol mental model

Topic 13.2: MCP server and client capabilities

Topic 13.3: Security and enterprise use of MCP

Module 14: LlamaIndex And Data-Centric GenAI Systems

Topic 14.1: Data ingestion and indexing in LlamaIndex

Topic 14.2: Query engines, retrievers, and workflows

Topic 14.3: Document AI and knowledge-heavy applications

Module 15: ADK And OpenAI Agents SDK

Topic 15.1: Google ADK graph and runtime model

Topic 15.2: OpenAI Agents SDK patterns

Topic 15.3: Runtime comparison and selection

Module 16: Multi-Agent, Human-In-The-Loop, And Long-Lived Systems

Topic 16.1: Coordination patterns for multi-agent systems

Topic 16.2: Human-in-the-loop and approvals

Topic 16.3: Memory and long-horizon execution

Module 17: Multimodal, Voice, And Realtime GenAI

Topic 17.1: Multimodal input and output fundamentals

Topic 17.2: Voice agents and streaming UX

Topic 17.3: Document AI and visual RAG

Module 18: DSPy, Fine-Tuning, Distillation, And Optimization

Topic 18.1: When prompting stops being enough

Topic 18.2: DSPy and program optimization

Topic 18.3: Fine-tuning, distillation, and model adaptation

Module 19: Capstones And Mastery Loops

Topic 19.1: Capstone A - production-grade RAG assistant

Topic 19.2: Capstone B - LangGraph plus MCP workflow agent

Topic 19.3: Capstone C - multimodal or document AI system

Module 20: Cost Engineering And Product Tradeoffs For GenAI

Topic 20.1: Token economics and usage analysis

Topic 20.2: Latency budgeting and pipeline design

Topic 20.3: Cost-quality-product decision frameworks

Module 21: GenAI Debugging Playbook

Topic 21.1: Failure taxonomy and triage

Topic 21.2: Layer-by-layer debugging workflow

Topic 21.3: Interview-grade diagnosis and incident reviews

Module 22: Portfolio Packaging And Hiring Signal Design

Topic 22.1: Architecture storytelling assets

Topic 22.2: Failure analysis and tradeoff documents

Topic 22.3: Hiring-facing packaging

Permanent Weekly Loops To Run Alongside The Modules

Completion Definition For True Mastery

What We Cover First When We Start Executing This Canon

Final Outcome

This document is the master roadmap for your GenAI journey.

It is intentionally focused on GenAI itself and excludes general software topics for now such as Python, TypeScript, Redis, Docker, Cloud Run, CI/CD, and frontend frameworks. Those matter later, but this canon is for mastering the GenAI core deeply enough that you stand out in the market.

Linked Documents

- [GenAI Document Index](#)
- [GenAI 10-12 Hour Weekly Execution Plan](#)
- [GenAI Mini-Project Specifications](#)

How To Use This Canon

- Treat each module as a serious study block, not a checklist.
- The time estimates include first-pass learning, note making, and one recap pass.
- If you build while learning, add 25% to 40% more time.
- Use the market-ready track first if your goal is faster career impact. Use the full canon if your goal is deep mastery.
- Every two modules, ship something. This canon is only valuable if it becomes visible skill.
- Do not skip evaluation, safety, or retrieval engineering. Those are what separate toy demos from real systems.
- Track cost, latency, and failure modes from the beginning, not after the system becomes expensive or brittle.
- In every module, ask two questions: should this use GenAI at all, and if yes, what is the cheapest reliable version?
- The goal is not to memorize framework APIs. The goal is to understand the concepts strongly enough to choose, justify, and build the right GenAI system.

What This Canon Covers

- LLM mental models
- Transformer and model internals
- Prompting and structured generation
- Embeddings and semantic search
- Vector databases and retrieval systems
- RAG from baseline to advanced
- Evaluation, observability, and reliability
- Guardrails and GenAI safety
- Agents, tool use, memory, and planning
- LangChain, LangGraph, MCP, LlamaIndex, ADK, and OpenAI Agents SDK
- Multi-agent systems, multimodal systems, voice, realtime, DSPy, and model adaptation
- Cost engineering and product tradeoff thinking
- Debugging playbooks across model, retrieval, prompt, tool, and orchestration layers
- Portfolio packaging that converts projects into hiring signals
- Capstones that prove real mastery

Execution Modes

- Market-ready acceleration track: Focus on Modules 1, 3, 4, 5, 6, 8, 20, 7, 9, 10, 11, 12, 13, and 21. This is the highest-ROI path for standing out quickly.
- Hiring-ready track: Complete the market-ready acceleration track, ship at least one capstone slice from Module 19, and complete Module 22.
- Full mastery track: Complete all modules with the build cadence and weekly loops.

Time Model

- Original foundation plus framework track: Modules 1-13, about 397 hours
- Market-ready acceleration track: selected modules, about 413 hours
- Hiring-ready track with one packaged capstone: about 449 hours
- Full mastery canon: Modules 1-22, about 689 hours
- At 10-12 hours per week, the hiring-ready track is roughly 9 to 11 months
- At 10-12 hours per week, the full mastery canon is roughly 13 to 16 months
- At 15-18 hours per week, the hiring-ready track is roughly 6 to 7 months
- At 15-18 hours per week, the full mastery canon is roughly 10 to 11 months

Mandatory Build Cadence

These builds are part of the roadmap, not optional extras.

- After **Module 1** and **Module 3**: Build **Project 1: Structured Output Assistant**.
- After **Module 4** and **Module 5**: Build **Project 2: Semantic Search Lab**.
- After **Module 6** and **Module 8**: Build **Project 3: Baseline RAG Assistant With Citations**.
- After **Module 7** and **Module 9**: Build **Project 4: Advanced RAG With Reranking And Guardrails**.
- After **Module 10** and **Module 12**: Build **Project 5: LangGraph Workflow Agent**.
- After **Module 13** and **Module 20**: Build **Project 6: MCP-Enabled Workflow With Cost And Latency Budget**.
- After **Module 14** and **Module 15**: Build **Project 7: Data-Heavy Assistant And Framework Selection Memo**.
- After **Module 16** and **Module 17**: Build **Project 8: Long-Lived Multimodal Or Human-In-The-Loop System**.
- After **Module 18** and **Module 21**: Build **Project 9: Optimization Or Debugging Case Study**.
- After **Module 19** and **Module 22**: Build **Project 10: Hiring-Grade Capstone Asset Pack**.

Phase Map

Phase	Modules	Focus	Total Time
Phase I	1-5	Foundations, prompting, embeddings, vector search	128h
Phase II	6-9	RAG, evaluation, safety, and reliability	138h
Phase III	10-13	Agents, LangChain, LangGraph, MCP	131h
Phase IV	14-18	Data-heavy systems, runtimes, multimodal, optimization	158h
Phase V	19-22	Capstones, cost, debugging, and portfolio packaging	134h

Recommended Market-Strong Stack Hidden Inside This Canon

By the end of this roadmap, the strongest GenAI identity you should be able to claim is:

- LangGraph for orchestration
 - MCP for tool and context interoperability
 - RAG and retrieval engineering for knowledge systems
 - Postgres/pgvector, Qdrant, and Chroma concepts for vector storage decisions
 - LangChain for integration speed
 - LlamaIndex for data-heavy and document-heavy systems
 - ADK and OpenAI Agents SDK as important modern runtimes to compare and understand
 - Strong evaluation, safety, reliability, and cost discipline
 - A repeatable debugging mindset across every GenAI layer
-

Module 1: GenAI Landscape And Mental Models

Module time: 18h

Why this module matters: Before touching frameworks, you need a precise mental model of what a GenAI system actually is.

Topic 1.1: GenAI system taxonomy and vocabulary

Topic time: 4h

- Foundation model vs instruct model vs reasoning-oriented model - 45m
- Assistant vs copilot vs workflow vs agent - 45m
- Hosted vs open-weight vs self-hosted model ecosystems - 60m
- Tokens, context windows, latency, throughput, and cost basics - 90m

Topic 1.2: Anatomy of a GenAI application

Topic time: 6h

- Model layer, prompt layer, tool layer, retrieval layer - 90m
- Memory, knowledge grounding, and feedback loops - 90m
- Evaluation, tracing, and safety as system components - 90m
- Reliability, latency, and cost as product constraints - 90m

Topic 1.3: Failure modes and thinking patterns

Topic time: 8h

- Hallucination, omission, shallow retrieval, and overconfident answers - 2h
- Prompt brittleness, hidden state, and context overload - 2h
- Tool misuse, stale knowledge, and permission blind spots - 2h
- Root-cause decomposition: model bug vs retrieval bug vs tool bug vs orchestration bug - 2h

Module checkpoint:

- Explain the full anatomy of a GenAI system without using vague terms.
 - Distinguish cleanly between workflow automation, RAG, and agentic behavior.
 - Diagnose a bad answer by mapping it to the correct failure layer.
-

Module 2: Transformer And Modern LLM Internals

Module time: 30h

Why this module matters: This is the theory layer that prevents you from becoming dependent on tutorials and cargo-cult explanations.

Topic 2.1: Text processing, tokens, and context

Topic time: 6h

- Text normalization, segmentation, and token boundaries - 90m
- BPE and SentencePiece intuition - 90m
- Positional information, context windows, and truncation risks - 90m
- Token budgeting for prompts, retrieval context, and tool results - 90m

Topic 2.2: Transformer mechanics

Topic time: 12h

- Embeddings, self-attention, heads, and layers - 3h
- Feed-forward blocks, residual connections, and normalization - 3h
- Why attention works, where it breaks, and long-context variants - 3h
- Inference behavior: KV cache, batching, latency, and throughput - 3h

Topic 2.3: From pretraining to instruction following

Topic time: 12h

- Next-token prediction and what pretraining actually teaches - 3h
- SFT, alignment, and preference optimization concepts - 3h
- Tool-use and reasoning behavior as trained capabilities - 3h
- Why smaller tuned models can beat larger untuned models on narrow tasks - 3h

Module checkpoint:

- Explain attention clearly to a beginner and to an engineer.
 - Reason about why a model fails under long context or tool-heavy workloads.
 - Describe how pretraining, SFT, and alignment shape model behavior.
-

Module 3: Prompting And Structured Generation

Module time: 28h

Why this module matters: Prompting is still necessary, but serious practitioners move from clever prompts to structured generation systems.

Topic 3.1: Prompt design patterns

Topic time: 8h

- Roles, objectives, constraints, and examples - 2h
- Zero-shot, few-shot, chain-of-thought, and self-consistency patterns - 2h
- Decompose, critique, reflect, verify patterns - 2h
- Prompt templating and variable hygiene - 2h

Topic 3.2: Structured output and schema-driven generation

Topic time: 10h

- JSON, XML, Markdown, and typed output strategies - 2.5h
- Function calling and schema-based tool invocation - 2.5h
- Validation, retry, repair, and correction loops - 2.5h
- Deterministic extraction vs creative generation tradeoffs - 2.5h

Topic 3.3: Prompt debugging and prompt systems

Topic time: 10h

- Prompt diffing, experiment logs, and version discipline - 2.5h
- Instruction ordering and context packing strategies - 2.5h
- Failure triage: ambiguity, overload, contradiction, leakage - 2.5h
- System prompt, developer prompt, and user prompt boundaries - 2.5h

Module checkpoint:

- Design prompts that are auditable, testable, and reproducible.
 - Use schemas and validation instead of trusting free-form outputs.
 - Identify when prompting is the wrong layer to fix the problem.
-

Module 4: Embeddings And Semantic Representations

Module time: 24h

Why this module matters: If you want to build RAG, search, memory, or retrieval systems, embeddings are unavoidable.

Topic 4.1: Embedding concepts and vector geometry

Topic time: 8h

- What embeddings capture semantically and what they do not - 2h
- Cosine similarity vs dot product vs Euclidean distance - 2h
- Neighborhoods, clustering, and semantic drift - 2h
- Polysemy, multilinguality, and domain-shift limitations - 2h

Topic 4.2: Embedding model selection and evaluation

Topic time: 8h

- General-purpose vs domain-tuned embedding models - 2h
- Dimensions, latency, cost, and multilingual support - 2h
- Benchmarking with retrieval metrics instead of vibes - 2h
- Re-embedding strategies, versioning, and migration planning - 2h

Topic 4.3: Embedding pipelines and chunk representations

Topic time: 8h

- Chunk-level vs section-level vs document-level embeddings - 2h
- Query embeddings vs passage embeddings - 2h
- Metadata enrichment, titles, summaries, and hypothetical questions - 2h
- Refresh policies, backfills, and embedding drift management - 2h

Module checkpoint:

- Choose an embedding model with task-fit reasoning rather than brand preference.
 - Explain similarity metrics and when they matter.
 - Describe how bad chunk representation poisons downstream retrieval.
-

Module 5: Vector Search And Vector Datastores

Module time: 28h

Why this module matters: This is where embeddings become usable systems.

Topic 5.1: Similarity search fundamentals

Topic time: 8h

- Exact search vs approximate nearest neighbor search - 2h
- HNSW and IVFFlat intuition - 2h
- Recall vs latency vs memory tradeoffs - 2h
- Dense, sparse, and late-interaction retrieval basics - 2h

Topic 5.2: Vector database ecosystem

Topic time: 10h

- Chroma for local experimentation and prototypes - 2h
- pgvector for Postgres-native vector search - 2.5h
- Qdrant, Pinecone, and dedicated vector engines - 2.5h
- Multitenancy, namespaces, and metadata filters - 3h

Topic 5.3: Filtering, hybrid retrieval, and scale tradeoffs

Topic time: 10h

- Metadata filtering and partitioning patterns - 2.5h
- Hybrid dense plus sparse search designs - 2.5h
- Reranking after retrieval and its quality impact - 2.5h
- Index maintenance, cold data, deletes, and refresh costs - 2.5h

Module checkpoint:

- Explain why Chroma is good for learning but not always the final production choice.
 - Compare pgvector and dedicated vector engines without hype.
 - Reason about ANN quality tradeoffs in a business context.
-

Module 6: RAG Foundations

Module time: 34h

Why this module matters: RAG is still one of the highest-value GenAI skills in the market, but only when done properly.

Topic 6.1: Ingestion and preprocessing

Topic time: 10h

- Source inventory and content-quality audits - 2.5h
- Parsing PDFs, HTML, docs, and knowledge bases - 2.5h
- Chunking strategies: fixed, semantic, recursive, section-aware - 2.5h
- Metadata design: source, section, freshness, permissions - 2.5h

Topic 6.2: Retrieval pipeline basics

Topic time: 12h

- Query embedding and top-k retrieval flow - 3h
- Context packing and prompt stuffing basics - 3h
- Citation mapping and source traceability - 3h
- Common baseline RAG failures and debugging habits - 3h

Topic 6.3: Answer generation with citations

Topic time: 12h

- Grounded answer prompting - 3h
- Refusal behavior when evidence is insufficient - 3h
- Citation formatting, provenance, and source quoting - 3h
- Separating evidence from speculation and reasoning - 3h

Module checkpoint:

- Design a clean end-to-end baseline RAG system from raw documents to cited answers.
 - Explain why chunking and metadata often matter more than model swapping.
 - Refuse confidently when the evidence does not support an answer.
-

Module 7: Advanced Retrieval Engineering

Module time: 40h

Why this module matters: This is the module that turns simple RAG knowledge into market-relevant retrieval engineering.

Topic 7.1: Chunking, metadata, and hierarchical retrieval

Topic time: 12h

- Parent-child retrieval patterns - 3h
- Document hierarchy and section graph modeling - 3h
- Metadata-driven recall improvements - 3h
- Chunk overlap, redundancy, and context compaction - 3h

Topic 7.2: Query transformation and reranking

Topic time: 14h

- Query rewriting and expansion strategies - 3.5h
- Multi-query retrieval and fusion - 3.5h
- Cross-encoder and LLM reranking - 3.5h
- Reciprocal rank fusion and late fusion - 3.5h

Topic 7.3: Advanced RAG patterns

Topic time: 14h

- HyDE, self-RAG, and agentic retrieval patterns - 3.5h
- Multi-hop retrieval and decomposition - 3.5h
- Knowledge graph and GraphRAG fundamentals - 3.5h
- Conversation-aware and personalized retrieval - 3.5h

Module checkpoint:

- Improve retrieval quality using retrieval techniques, not only better prompts.
 - Explain when reranking is mandatory.
 - Compare baseline RAG, multi-hop RAG, and GraphRAG without confusing them.
-

Module 8: Evaluation, Observability, And Experimentation

Module time: 36h

Why this module matters: Real GenAI engineering is impossible without evaluation discipline.

Topic 8.1: Retrieval and generation metrics

Topic time: 12h

- Recall@k, MRR, NDCG, and hit rate - 3h
- Groundedness, faithfulness, and citation accuracy - 3h
- Task success vs answer polish - 3h
- Latency and cost as first-class quality metrics - 3h

Topic 8.2: Test sets, judges, and regression systems

Topic time: 12h

- Golden sets and annotation design - 3h
- LLM-as-judge patterns and failure modes - 3h
- Pairwise evals, ablations, and experiment structure - 3h
- Regression suites for prompts, retrieval, and tools - 3h

Topic 8.3: Tracing and production observability

Topic time: 12h

- Request traces, spans, and state inspection - 3h
- Capturing prompts, contexts, tool calls, and model outputs - 3h
- Human feedback collection and error labeling - 3h
- Closing the loop from trace to system improvement - 3h

Module checkpoint:

- Build an evaluation story for any GenAI system you discuss.
 - Measure retrieval and generation separately.
 - Explain how tracing leads to concrete system changes.
-

Module 9: Safety, Guardrails, And Reliability

Module time: 28h

Why this module matters: Market demand is shifting toward systems that are safe, controllable, and trustworthy.

Topic 9.1: Input and output safety

Topic time: 8h

- Jailbreaks, prompt injection, and policy bypass basics - 2h
- Output filtering, moderation, and policy shaping - 2h
- Structured safety checks and approval gates - 2h
- Intent classification and risk-tiered routing - 2h

Topic 9.2: Tool and retrieval security

Topic time: 10h

- Retrieval poisoning and data exfiltration risks - 2.5h
- Tool permissioning and least-privilege design - 2.5h
- Secret exposure and action-confirmation patterns - 2.5h
- Tenant isolation and permission-aware retrieval - 2.5h

Topic 9.3: Reliability engineering for LLM apps

Topic time: 10h

- Timeouts, retries, and fallback-model strategies - 2.5h
- Idempotency and side-effect control - 2.5h
- Human escalation and graceful degradation - 2.5h
- Reliability budgets for quality, latency, and cost - 2.5h

Module checkpoint:

- Explain why prompt injection is not just a prompt problem.
 - Design a safe tool-using assistant with explicit approval boundaries.
 - Talk about reliability using the language of engineering, not hope.
-

Module 10: Agent Fundamentals

Module time: 32h

Why this module matters: Agents are useful, but only once you understand when they are justified and how they fail.

Topic 10.1: What agents are and are not

Topic time: 8h

- Agent vs chain vs workflow vs assistant - 2h
- When deterministic workflows beat agent loops - 2h
- The agent loop: observe, think, act, update - 2h
- Common anti-patterns in agent design - 2h

Topic 10.2: Tool use, planning, and memory

Topic time: 12h

- Tool schemas and tool selection behavior - 3h
- Planning styles: reactive, plan-and-execute, hierarchical - 3h
- Short-term vs long-term memory - 3h
- Context compaction and summary memory - 3h

Topic 10.3: Agent architectures and failure handling

Topic time: 12h

- Single-agent with tools - 3h
- Supervisor-worker and router patterns - 3h
- Recovery from tool errors, loops, and dead ends - 3h
- Evaluating full trajectories, not just final responses - 3h

Module checkpoint:

- Decide correctly when not to use an agent.
 - Explain planning, memory, and tool use as separate concerns.
 - Diagnose agent failures as control-flow problems, not just model problems.
-

Module 11: LangChain Core

Module time: 30h

Why this module matters: LangChain remains the fastest way to understand many common GenAI building blocks and integrations.

Topic 11.1: LangChain core abstractions

Topic time: 10h

- Models, messages, prompts, and outputs - 2.5h
- Tools, retrievers, and document abstractions - 2.5h
- Runnable and composition patterns - 2.5h
- Integration strategy without over-coupling your app - 2.5h

Topic 11.2: Retrieval, tools, and agents in LangChain

Topic time: 10h

- Building a clean RAG flow - 2.5h
- Tool wrapping and schema design - 2.5h
- Prebuilt agents vs custom control logic - 2.5h
- Streaming, callbacks, and trace-friendly design - 2.5h

Topic 11.3: Production use of LangChain

Topic time: 10h

- Keeping prompts and configs out of spaghetti code - 2.5h
- Using LangSmith for traces and evals - 2.5h
- Migration boundaries between LangChain and LangGraph - 2.5h
- When LangChain should stay as integration glue only - 2.5h

Module checkpoint:

- Use LangChain for speed without letting it own your architecture.
 - Explain where LangChain ends and orchestration begins.
 - Keep business logic independent from framework convenience layers.
-

Module 12: LangGraph Mastery

Module time: 45h

Why this module matters: This is the most important orchestration framework in your target stack.

Topic 12.1: Graph mental models and state design

Topic time: 12h

- State graphs, nodes, edges, and transitions - 3h
- Designing minimal but expressive state - 3h
- Conditional routing and deterministic checks - 3h
- Subgraphs and reusable workflow fragments - 3h

Topic 12.2: Durable execution, persistence, and interrupts

Topic time: 15h

- Checkpointing and resumability - 3.75h
- Human-in-the-loop interrupts and approvals - 3.75h
- Error recovery, replay, and restartability - 3.75h
- Long-running workflows and evolving state - 3.75h

Topic 12.3: Production graph patterns

Topic time: 18h

- Research-agent graph patterns - 4.5h
- Retrieval-enriched workflow graphs - 4.5h
- Multi-actor graphs with specialist nodes - 4.5h
- Testing, tracing, and optimizing graph behavior - 4.5h

Module checkpoint:

- Model long-running, stateful workflows as explicit graphs.
 - Add human approval and recovery points without destroying flow clarity.
 - Explain why LangGraph is stronger than simple agent loops for serious systems.
-

Module 13: Model Context Protocol (MCP)

Module time: 24h

Why this module matters: MCP is becoming a standard interface for tool and context interoperability across clients and runtimes.

Topic 13.1: MCP protocol mental model

Topic time: 6h

- Why MCP exists and what it standardizes - 1.5h
- Client, server, transport, and capability model - 1.5h
- Tools, resources, and prompts in MCP - 1.5h
- MCP vs direct APIs and SDK-specific tools - 1.5h

Topic 13.2: MCP server and client capabilities

Topic time: 10h

- Designing useful MCP tools - 2.5h
- Exposing data as resources vs tools - 2.5h
- Authentication, authorization, and multitenancy - 2.5h
- Integrating MCP into agent frameworks - 2.5h

Topic 13.3: Security and enterprise use of MCP

Topic time: 8h

- Approval flows and dangerous-action containment - 2h
- Auditing and policy enforcement - 2h
- Standardizing internal enterprise tool access - 2h
- Comparing MCP usage across assistants, IDEs, and runtimes - 2h

Module checkpoint:

- Explain MCP clearly as a protocol, not a buzzword.
 - Decide whether a capability should be an MCP tool, resource, or plain API.
 - Reason about enterprise MCP design with security in mind.
-

Module 14: LlamaIndex And Data-Centric GenAI Systems

Module time: 28h

Why this module matters: LlamaIndex becomes especially valuable once your problem is deeply tied to data ingestion, documents, and retrieval workflows.

Topic 14.1: Data ingestion and indexing in LlamaIndex

Topic time: 10h

- Loaders, readers, and connectors - 2.5h
- Parsing, nodes, and document representation - 2.5h
- Index types and retrieval abstractions - 2.5h
- Data-centric pipeline design choices - 2.5h

Topic 14.2: Query engines, retrievers, and workflows

Topic time: 10h

- Query engines and response synthesis - 2.5h
- Retriever customization and fusion - 2.5h
- Workflow orchestration in data-heavy applications - 2.5h
- When LlamaIndex beats generic frameworks for knowledge tasks - 2.5h

Topic 14.3: Document AI and knowledge-heavy applications

Topic time: 8h

- Document parsing and structure extraction concepts - 2h
- Tables, forms, and structured extraction workflows - 2h
- Knowledge assistants and research copilots - 2h
- Evaluation for document understanding systems - 2h

Module checkpoint:

- Explain when LlamaIndex is the right fit compared with LangChain or LangGraph.
 - Describe document-heavy system design using ingestion and indexing vocabulary.
 - Reason about structured extraction as more than plain text retrieval.
-

Module 15: ADK And OpenAI Agents SDK

Module time: 30h

Why this module matters: These are important modern runtimes to understand after you already have strong fundamentals.

Topic 15.1: Google ADK graph and runtime model

Topic time: 10h

- ADK agent model and tool patterns - 2.5h
- Graph workflows and routing - 2.5h
- Sessions, state, and evaluation concepts - 2.5h
- When ADK is a better fit than LangGraph - 2.5h

Topic 15.2: OpenAI Agents SDK patterns

Topic time: 10h

- Agent, runner, tools, and handoffs - 2.5h
- Guardrails and sessions - 2.5h
- MCP integration and sandbox agents - 2.5h
- Realtime and voice-oriented pathways - 2.5h

Topic 15.3: Runtime comparison and selection

Topic time: 10h

- LangGraph vs ADK vs OpenAI Agents SDK - 2.5h
- Lock-in, control, observability, and runtime tradeoffs - 2.5h
- Team skill fit and ecosystem maturity - 2.5h
- Building a framework-selection rubric - 2.5h

Module checkpoint:

- Compare agent runtimes with engineering arguments rather than fandom.
 - Choose a runtime based on workflow shape, not vendor popularity.
 - Explain why LangGraph remains your anchor even after learning alternatives.
-

Module 16: Multi-Agent, Human-In-The-Loop, And Long-Lived Systems

Module time: 34h

Why this module matters: This is where agent systems become genuinely operational instead of theatrical.

Topic 16.1: Coordination patterns for multi-agent systems

Topic time: 10h

- Manager-worker and router-specialist patterns - 2.5h
- Blackboard and shared-state coordination - 2.5h
- Debate, critique, and verifier patterns - 2.5h
- Why many multi-agent systems should stay single-agent - 2.5h

Topic 16.2: Human-in-the-loop and approvals

Topic time: 12h

- Approval checkpoints and reversible actions - 3h
- Confidence thresholds and escalation logic - 3h
- UX implications of human review - 3h
- Measuring intervention quality and operational cost - 3h

Topic 16.3: Memory and long-horizon execution

Topic time: 12h

- Episodic, semantic, and procedural memory concepts - 3h
- Session memory, summary memory, and retrieval memory - 3h
- Memory freshness, drift, and forgetting strategies - 3h
- Long-running task decomposition and checkpoint strategy - 3h

Module checkpoint:

- Explain why many “multi-agent” demos are just bad workflow design.
 - Build approval boundaries into long-running systems.
 - Treat memory as a system design problem, not a magical feature.
-

Module 17: Multimodal, Voice, And Realtime GenAI

Module time: 30h

Why this module matters: The market is shifting from text-only systems toward multimodal and realtime experiences.

Topic 17.1: Multimodal input and output fundamentals

Topic time: 10h

- Images, documents, audio, and video as inputs - 2.5h
- OCR vs VLM reasoning tradeoffs - 2.5h
- Multimodal prompt construction - 2.5h
- Artifact generation and multimodal outputs - 2.5h

Topic 17.2: Voice agents and streaming UX

Topic time: 10h

- STT to agent to TTS pipeline - 2.5h
- Turn-taking, interruption, and latency targets - 2.5h
- Realtime session state and tool use - 2.5h
- Safety and observability for live voice systems - 2.5h

Topic 17.3: Document AI and visual RAG

Topic time: 10h

- Tables, charts, diagrams, and layout-aware retrieval - 2.5h
- Page-level vs block-level grounding - 2.5h
- UI and screenshot understanding use cases - 2.5h
- End-to-end multimodal evaluation - 2.5h

Module checkpoint:

- Explain the difference between OCR pipelines and multimodal reasoning systems.
 - Reason about realtime voice systems using latency and turn-taking constraints.
 - Discuss visual grounding as a retrieval and evaluation problem.
-

Module 18: DSPy, Fine-Tuning, Distillation, And Optimization

Module time: 36h

Why this module matters: This is the layer you reach when prompts and retrieval tuning are no longer enough.

Topic 18.1: When prompting stops being enough

Topic time: 10h

- Diagnosing prompt ceiling vs data ceiling vs model ceiling - 2.5h
- Systematic error analysis for model adaptation - 2.5h
- Synthetic data generation and curation - 2.5h
- ROI analysis for optimization work - 2.5h

Topic 18.2: DSPy and program optimization

Topic time: 12h

- Signatures, modules, and declarative AI programs - 3h
- Optimizers for few-shot and instruction search - 3h
- Evaluating optimized programs honestly - 3h
- Where DSPy fits relative to framework-centric stacks - 3h

Topic 18.3: Fine-tuning, distillation, and model adaptation

Topic time: 14h

- SFT, PEFT, LoRA, and adapter mental models - 3.5h
- Distillation and teacher-student pipelines - 3.5h
- Fine-tuning for extraction, classification, and domain adaptation - 3.5h
- Evaluation, rollback, and maintenance of tuned models - 3.5h

Module checkpoint:

- Know when optimization is justified and when it is wasted effort.
 - Explain DSPy as optimization of AI programs rather than prompt tweaking.
 - Describe fine-tuning with realistic maintenance expectations.
-

Module 19: Capstones And Mastery Loops

Module time: 72h

Why this module matters: Without capstones, all of this remains academic.

Topic 19.1: Capstone A - production-grade RAG assistant

Topic time: 20h

- Problem framing, source inventory, and evaluation targets - 4h
- Retrieval design: chunking, embeddings, vector store, reranking - 6h
- Answer generation, citation policy, and guardrails - 5h
- Evaluation loop, failure analysis, and architecture review - 5h

Topic 19.2: Capstone B - LangGraph plus MCP workflow agent

Topic time: 24h

- Workflow selection and graph design - 5h
- Tool surface and MCP integration plan - 7h
- Interrupts, approvals, and recovery design - 6h
- Trace review, optimization, and architecture defense - 6h

Topic 19.3: Capstone C - multimodal or document AI system

Topic time: 28h

- Use-case scoping and modality selection - 5h
- Retrieval or understanding pipeline design - 8h
- Evaluation rubric and failure-mode mapping - 7h
- Architecture evidence collection, demo narrative, and system defense - 8h

Module checkpoint:

- Present three serious GenAI systems with architecture-level confidence.
 - Defend model choices, retrieval strategy, eval design, and safety design.
 - Show employers that you understand systems, not just prompts.
-

Module 20: Cost Engineering And Product Tradeoffs For GenAI

Module time: 24h

Why this module matters: Strong GenAI engineers do not just make systems work. They make them affordable, fast enough, and worth deploying.

Topic 20.1: Token economics and usage analysis

Topic time: 8h

- Prompt token accounting and token growth across turns - 2h
- Retrieval context expansion and tool output explosion - 2h
- Cost per request, cost per session, and cost per successful task - 2h
- Logging and reviewing token consumption by system layer - 2h

Topic 20.2: Latency budgeting and pipeline design

Topic time: 8h

- End-to-end latency decomposition across retrieval, reranking, tools, and generation - 2h
- Streaming, batching, concurrency, and timeout budgets - 2h
- Should you rerank or increase top-k: tradeoff reasoning - 2h
- Should you compress context or use a larger model: tradeoff reasoning - 2h

Topic 20.3: Cost-quality-product decision frameworks

Topic time: 8h

- When GenAI is justified vs when deterministic logic is better - 2h
- Model routing, fallback tiers, and dynamic quality tiers - 2h
- Retrieval cost vs generation cost vs engineering cost - 2h
- ROI framing for product, platform, and enterprise systems - 2h

Module checkpoint:

- Explain token, retrieval, and reranking costs as one system budget.
 - Defend whether to rerank, increase top-k, compress context, or route to a different model.
 - Explain when the right product decision is to use less GenAI, not more.
-

Module 21: GenAI Debugging Playbook

Module time: 22h

Why this module matters: Senior engineers are judged by how they debug messy failures across layers, not by how confidently they talk about tools.

Topic 21.1: Failure taxonomy and triage

Topic time: 6h

- Retrieval issue vs prompt issue vs model limitation vs tool issue vs orchestration failure - 1.5h
- Symptom-based diagnosis patterns - 1.5h
- Reproducibility, fixtures, and failure isolation - 1.5h
- Building a first-pass triage checklist - 1.5h

Topic 21.2: Layer-by-layer debugging workflow

Topic time: 8h

- Inspecting retrieval candidates, chunk quality, and missing evidence - 2h
- Auditing prompts, context order, and schema constraints - 2h
- Tracing tool calls, agent trajectories, and graph state - 2h
- Distinguishing model limitations from orchestration mistakes - 2h

Topic 21.3: Interview-grade diagnosis and incident reviews

Topic time: 8h

- Writing root-cause summaries and remediation plans - 2h
- Designing targeted experiments to disconfirm a hypothesis - 2h
- Rollback decisions, fallback paths, and safe mitigations - 2h
- Explaining a failure clearly in interviews and design reviews - 2h

Module checkpoint:

- Diagnose whether a failure is caused by retrieval, prompt, model, tool, or orchestration.
 - Use traces and controlled experiments instead of guesswork.
 - Explain a GenAI incident with senior-level clarity and concrete remediation steps.
-

Module 22: Portfolio Packaging And Hiring Signal Design

Module time: 16h

Why this module matters: Good projects do not automatically become strong hiring signals. Packaging is what turns work into evidence.

Topic 22.1: Architecture storytelling assets

Topic time: 5h

- Architecture diagrams that show system layers clearly - 1.5h
- README structure for GenAI projects - 1h
- Demo script design and narrative discipline - 1.5h
- What to show first to recruiters, hiring managers, and engineers - 1h

Topic 22.2: Failure analysis and tradeoff documents

Topic time: 5h

- Failure analysis writeups and postmortem-lite documents - 1.25h
- Tradeoff justification: why this model, retrieval strategy, and workflow - 1.25h
- Evaluation reports that show before-vs-after improvement - 1.25h
- Rejected alternatives and why they lost - 1.25h

Topic 22.3: Hiring-facing packaging

Topic time: 6h

- Resume bullets grounded in measurable system outcomes - 1.5h
- Project case-study pages and portfolio summaries - 1.5h
- Interview walkthroughs for architecture, failures, and tradeoffs - 1.5h
- Open-source hygiene, visuals, and presentation quality - 1.5h

Module checkpoint:

- Package each serious project with an architecture diagram, failure analysis document, and tradeoff justification.
 - Present your work as systems engineering evidence, not feature hype.
 - Make one capstone interview-ready without relying on live explanation alone.
-

Permanent Weekly Loops To Run Alongside The Modules

These are not included in the module hour counts. Run them continuously.

- Paper digestion and note synthesis - 2h per week
- Eval journal: write down one failure and its probable root cause - 1h per week
- Cost and latency review: inspect one expensive or slow path and propose one fix - 45m per week
- Framework change log review for LangGraph, MCP, LlamaIndex, ADK, and OpenAI Agents SDK - 1h per week
- Retrieval quality review: inspect at least five bad retrieval cases - 1h per week
- Debugging drill: classify one failure by system layer and write the shortest reproducible explanation - 45m per week
- Explain-one-concept-out-loud drill - 30m per week

Completion Definition For True Mastery

You can say you have real GenAI mastery when you can do all of the following without bluffing:

- Explain model behavior, retrieval behavior, and agent behavior as separate system layers.
- Design RAG systems beyond vector search slogans.
- Choose between Chroma, pgvector, Qdrant, and managed vector platforms using tradeoffs.
- Use LangChain as a helper, LangGraph as an orchestrator, and MCP as a protocol layer.
- Explain when LlamaIndex, ADK, or OpenAI Agents SDK is the better fit.
- Build evaluation into the system from day one.
- Budget token usage, latency, retrieval cost, and generation cost with intent.
- Diagnose failures by layer using a repeatable debugging playbook.
- Treat safety, permissions, and approvals as architecture, not add-ons.
- Discuss multimodal, realtime, and optimization paths with technical maturity.
- Package at least one project with architecture, failure analysis, and tradeoff evidence.

What We Cover First When We Start Executing This Canon

Recommended order of execution:

1. Module 1
2. Module 3
3. Module 4
4. Module 5
5. Module 6
6. Module 8
7. Module 20
8. Module 7
9. Module 9
10. Module 10
11. Module 11
12. Module 12
13. Module 13
14. Module 21
15. Module 2
16. Module 14
17. Module 15
18. Module 16
19. Module 17
20. Module 18
21. Module 19
22. Module 22

Note on the sequence:

- Module 2 is still important, but it is intentionally delayed after the first market-critical build layers so you gain systems leverage early without skipping theory forever.
- Module 8 is intentionally pulled ahead of the full advanced retrieval module because evaluation habits should start earlier than most people think.
- Module 20 is pulled forward because cost and latency decisions should be learned before systems become expensive and slow.
- Module 21 is pulled forward because debugging maturity is a major senior-level differentiator.
- LangGraph and MCP are treated as the core differentiators of the modern stack.
- Chroma is learned as part of the vector ecosystem, but not treated as the final answer for every production system.
- Module 22 is last in the list but should run in parallel once the first serious project exists.

Final Outcome

If you complete this canon properly, your market identity should be:

A GenAI systems engineer who understands retrieval, orchestration, evaluation, safety, cost, debugging, and modern agent runtimes deeply enough to design, defend, and package real-world systems rather than demo apps.